

Design of Combinational Logic Circuits Using a New Metaheuristic Algorithm

Walid Mohamed Aly

Technical & Vocational Institute, Arab Academy for Science, Technology and Maritime Transport, Alexandria, Egypt

Abstract — Combinational logic circuit design is a dual problem, the design should be 100% functional and in the same time an optimum functional design should utilize minimum numbers of gates.

This paper proposes a new metaheuristic based algorithm for designing relatively large combinational logic circuits using minimum number of gates. The algorithm combines the exploration power of random search to the intelligence powers of heuristic search and the essence of Tabu search to present a methodology that can be applied efficiently to solve the design problem. The validity of the proposed algorithm is supported by its applying to 4 different problems of 12 inputs.

Index Terms — Search methods, Logic design, Artificial intelligence Intelligence, Combinational logic circuits, Design automation

I. INTRODUCTION

Any combinational logic circuit can be described using a switching logic expression, however it is essential to simplify this expression to its minimal form to reduce the cost of its realizing using gates, this ensures decreasing the cost and increasing the reliability of the circuit.

Several theorems of Boolean algebra-like the consensus theorem - can be used to perform this simplification however there are neither strict rules of the sequence by which these simplification rules should be applied nor there is an exact test procedure to realize that the minimum expression is reached.

Karnaugh map (k-map) method and Quine-McCluskey procedure[1] are used to overcome these drawbacks. Using k-map can easily be applied to minimize a logic expression for functions with three or four variables, and can still be applied to 5 or 6 variables. However for function with more than 6 variables, it's very difficult to apply k-map method as it would be very hard to spot the patterns needed by the solution in multidimensional space.

Quine-McCluskey procedure is used also to minimize logic expression and it is more practical to apply when problem variable are more than four.

The procedure involves:-

- Finding the essential prime implicants.
- Choosing minimal set of remaining prime implicant to cover the function.

Unfortunately the problem is still NP-hard and for a function of n variables the number of implicant can reach

$\frac{3^N}{N}$ [2], for $n=32$, the prime implicant may be about 600 trillion. Heuristic and metaheuristic techniques should be applied to this design problem when the numbers of computation is not practical and a good solution, although not guaranteed to be optimum is still satisfactory.

In this paper we present a new metaheuristic algorithm to form a procedure that can be applied to simplify a logic expression in a practical time and can be applied on relatively large combinational circuits.

The paper is designed as follows, the current section is the introductory section which gives a brief about the problem under consideration, section two discusses the general idea of metaheuristics with more concern on Tabu Algorithms.

Section three gives a fast survey on pervious work followed by section four which discusses the proposed algorithm followed with a section on test cases that are used to validate the algorithm, last section concludes the research.

The structure of the solution of the design problem using the new metaheuristic algorithm will be the sum of products (SOP), which is same the structure given by both k-maps and Quine-McCluskey procedures.

II. METAHEURISTICS

In real world, very hard computational problems exist where there are no exact applicable algorithm that is guaranteed to find the optimum solution in reasonable time and using reasonable resources. For these types of complex problems, heuristics and metaheuristics are usually used.

Metaheuristics are subset of the heuristics and are based on an artificial intelligence (AI) technique to develop its heuristic mechanism[3], however they are not guaranteed to find the optimum solution. Metaheuristics include simulated annealing, Tabu search(TS), Genetic algorithms and Swarm intelligence.

Tabu Search is a metaheuristic method originally proposed by Glover in 1986[4]. And as the founder of Tabu Search stated : “The overall approach is to avoid entrapment in cycles by forbidding or penalizing moves which take the solution, in the next iteration, to points in the solution space previously visited”.

To accomplish this, TS monitors the progress of the search, and uses this search *history* to actively modify the permitted neighbourhood solutions and forbidden the search in other neighbourhoods thus making them *Tabu*. Two major procedures are usually applied during Tabu search; Intensification and Diversification.

Intensification: The idea behind the concept of search intensification is that, as an intelligent human being would probably do, one should explore more thoroughly the portions of the search space that seem “promising” in order to make sure that the best solutions in these areas are indeed found.

Diversification: One of the main problems of all methods based on Local Search approaches, and this includes TS in spite of the beneficial impact of Tabu, is that they tend to be too “local”. They tend to spend most, if not all, of their time in a restricted portion of the search space. The negative consequence of this fact is that, although good solutions may be obtained, one may fail to explore the most interesting parts of the search space and thus end up with solutions that are still far from the optimal ones. *Diversification* is an algorithmic mechanism that tries to alleviate this problem by forcing the search into previously unexplored areas of the search space. It is usually based on some form of *long-term memory* of the search.

III. PREVIOUS WORK

The design of logic circuit is recognized –like any other design activity- as an intelligent act. Several researchers had tackled the problem using various metaheuristic methods. One of the frequent metaheuristic applied is the genetic algorithms. However as genetic algorithms are population based search, they seems to be difficult to solve a problem with large number of inputs as the size of chromosome will increase and computing time sharply rise[5].

In [6] Miller et al used genetic algorithms to evolve digital circuits, some of these circuits were the one bit, two bit adders with carry and the three, four bit adders. All of these circuits used the resources available on Xilinx 6216 FPGA device, using FPGA. Miller et al also built an Intrinsic evolvable hardware platform (IEHW) in which

the evolution process is accelerated dramatically compared to software or VHDL simulation [7].

The concept of improving the fitness function was also present in the work of Cecelia et al [8], they proposed a new concept for the fitness function used for the synthesis of combinational logic circuits as they used a fractional order dynamic fitness function. They claimed that the new concept revealed superior results in terms of speed and convergence to achieve a solution.

Cecelia et al also proposed a GA for designing combinational logic circuits[9] and applied this algorithm to design circuits like 2 to 1 multiplexer and four bit parity check, however their aim was not only to design functional circuits but also to have a minimum number of gates within these circuits.

A novel idea was presented by Hounsell et al[10] as they developed an evolutionary algorithm which has the ability to evolve and evaluate circuits in a Hardware Description Language (HDL) within a novel environment they termed as the virtual chip.

Some of the researcher applied other Artificial intelligence algorithms either by themselves or hybrid with evolutionary algorithms, as in [11] Moore et al. presented an algorithm that is used for designing combinational logic circuits, their algorithm were inspired from quantum evolution and *practical Swarm*, they claimed the superiority of their hybrid algorithm over other practical swarm algorithms and other evolutionary algorithms such as GA.

In [12], Coello et al showed a technique to design combinational logic circuits that produce circuits with less total number of gates than those produced with human designers, and what was interesting in their research is that they analyzed the solution produced by GA and noted that GA tried too decrease the number of gates as if it applying Demorgan Laws or any other Boolean simplification technique in a clear way that can be used to teach students what steps to follow to simplify further a certain solution.

IV. PROPOSED HEURISTIC ALGORITHM

The basic idea behind the proposed algorithm is conducting a point to point search, such that at each iteration, the candidate individual is simply created in SOP form using all the terms contained in a set with all the promising Terms (Gterm set).

A. Algorithm terminology

- Fitness (F): the fitness of each candidate solution is the percentage of correct bits produced by the solution compared to the required output bit pattern. a solution with 100 fitness is a functional solution.
- Complexity: total number of logic AND & OR operations in expression.
- Term: the product of variables entity in SOP expression.
- Individual: a candidate logic expression, stored together with its fitness value and complexity.
- Tabu term: a term is considered as a Tabu term if it outputs a 1 at any bit location that is required to be a "0", using the SOP form, the functional logic expression would never contains a Tabu term as the effect of a wrongful 1 can never be undone with other terms.
- Gterm set: if a term is not a Tabu term, it is added to the Gterm Set. This set is a sorted set where there are no identical terms as members of this set. Each term in the Gterm set has its own fitness.
- Term Fitness: the fitness value of each Term in the GTerm Set is based on the total number of ones that came from the Term on its evaluation, the higher the number of ones the higher the fitness. This Term Fitness serves as the *Heuristic* value for each Term.

B. Algorithm procedures

- RandomSearch: individuals are randomly created in SOP form and has its fitness evaluated, each term in a random expression is formed with 2 constrains :-
(C1): No problem variable should be redundant in same term.
(C2): No problem variable should exist in a term together with its complement.
- IndivEvaluate: This procedure evaluates each individual by evaluating all of its terms, as each term is evaluated, a check is done whether it would be considered as a Tabu Term or can be added to Gterm set.
- CleanGterm : For any 2 terms Term1,Term2 in Gterm set. If Term1 is totally contained in Term2. Then Term2 is eliminated from Gterm set.

For example if both $A'BC$ and $A'BCD$ exist in Gterm set, then this procedure will remove $A'BCD$ from the set. As definitely $A'BC$ has less complexity and will score a higher number of ones in the output than $A'BCD$.

- RemoveVariable: this procedure is applied to each term in Gterm set. It aims on improving the fitness of each term by removing a literal variable from it.
For example $B'FGH$ might be changed to $B'GH$ if the later has better fitness.
- UpdateIndividual: the candidate individual is created in SOP from all the terms in Gterm set.
- Versify: input variables that did not appear within terms in GTerm Set are reported. An individual is created using RandomSearch, however the creation is biased towards increasing the probability of appearance of those variables.

C. Algorithm Description

The main theme of the proposed algorithm is to break the problem of finding the required expression to a number of smaller problems of finding the correct terms. The memory of the search is the Gterm set, as long as it is updated the individual will it self be updated.

The steps of the heuristic algorithm can be summarized as follows :-

- 1- the problem is loaded as a text file that contains the series of 0s and 1s of the desired output.
- 2- Random search is conducted to gain knowledge about the problem and start populating the GTerm set.
- 3- Update the GTerm Set using the CleanGterm procedure.
- 4- An individual is created using the UpdateIndividual procedure
- 5- If the required fitness and complexity is reached, or if a certain number of iteration passed without improvement, search stops and go to step ten otherwise search continue to next step.
- 6- Intensification is conducted on the GTerm set using the procedure removeVariable.
- 7- Versification is conducted using the procedure Versify.
- 8- Random Search is conducted again -same like step 2 -as an exploration method.
- 9- Back to step three.
- 10- Solution declaration.

Figure 1 shows the basic flow chart for the proposed algorithm.

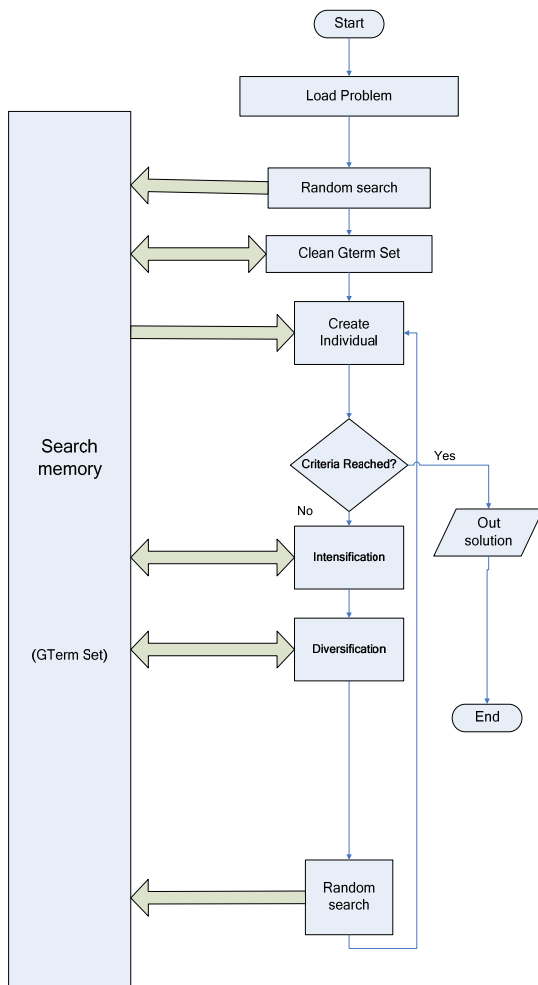


Fig.1. Algorithm Flowchart

D. Algorithm Analysis

- The complexity of the proposed algorithm can be described using the big O notation to be $O(2^n)$ where n is the number of input variables, as the main procedure used throughout the algorithm is IndivEvaluate.
- As any metaheuristic approach based on a stochastic process, the optimum solution is not guaranteed to be reached.
- Compared to evolutionary metaheuristics which are usually used for the same task. A point to point search

conducted by a Tabu search would probably need less time than population based metaheuristics.

- The proposed algorithm is inspired by Tabu Search by applying search with memory and spotting Tabu Solutions. however the search memory will hold only the non Tabu terms.
- Algorithm correctness can be analyzed using its general theme, random exploration simply test Terms to check whether they are problem implicant or not, procedure RemoveVariable aims to find the prime implicant. While procedure cleanGterm try to find essential prime implicant.

V. TEST CASES

A. Practical Results

Four different problems were used to test the validity of the algorithm, all have the same size of inputs which is 12 inputs. Table I reports these problems and the iteration at which the solution was found for each problem.

TABLE I
TEST PROBLEMS

Problem	Iteration
A.B'.C+D'.E'.H'.I'+J'.K.L.F'.G	18
A.B'.C+E'.H'+J'.L.F'.G	13
A.B'+C+D'.E'.H'.I.J'.K.L.F'.G	12
A.B'.C+A.D'.E'.H'.I+B'.J'.K.L.F'.G	43

Figure two and three show how both fitness and complexity of the candidate individual were evolving during solving the first problem.

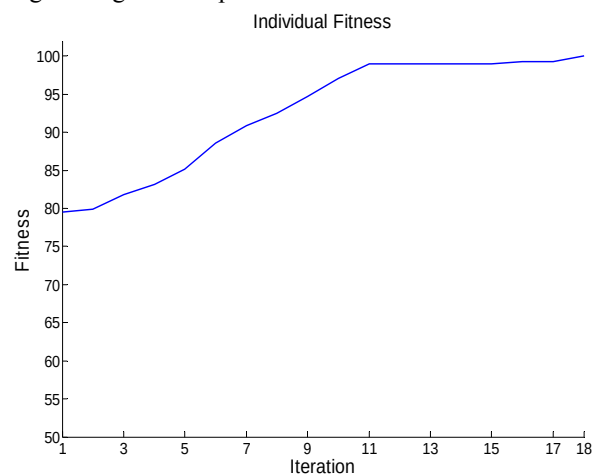


Fig.2 Individual Fitness vs. Iteration

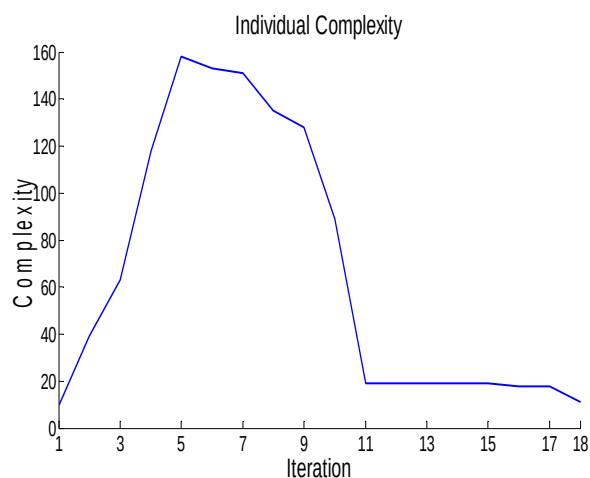


Fig.3. Individual Complexity vs. Iteration

B. Comments on Results

The proposed algorithm was able to solve all test cases, the first problem has moderate size terms and each input variable appeared only once.

In problem two, some of the input variables were “don’t care” and the proposed algorithm succeeded to eliminate them from the optimum solution declared, problem three was used to test the ability to find longer and shorter terms. Problem four- which was solved in the longest time- had a repetition case in a variables (A) and a compliment of a variable (B).

VI. CONCLUSION

A novel metaheuristic algorithm for automating the process of combinational logic circuit design is presented, The algorithm deploys a flexible structure while evolving the logic expression with the sum of product structure. The proposed algorithm is inspired by Tabu Search , special procedures were designed to bias the final solution to have only essential prime implicants.

Combining the exploration power of random search to the search memory usage of Tabu search together with the heuristic for measuring the fitness of each Term in the logic expression, had formed a valid algorithm for solving the design problem of relatively large combinational logic circuits.

REFERENCES

- [1] Fundamentals of Logic Design by Jr., Charles H. Roth, Publisher: Thomson-Engineering; 5 edition (June 11, 2003)
- [2] R. H. Katz, G. Borriello, *Contemporary Logic Design*, 2nd Ed., Pearson Prentice-Hall, Upper Saddle River, NJ, 2005
- [3] Russell S. and Norvig, P. *Artificial Intelligence: A Modern Approach*. Prentice Hall, New Jersey, 1995
- [4] F. Glover, future paths for integer programming and links with artificial intelligence", *Computers and Operations Research* 13 (1986),533-519
- [5] Chimiak and J. Kołodziejczyk and M. Opoka, „Synthesis of Combinational Binary Circuits based on G with Entropy Evaluation, Short Course on Evolutionary Algorithms in Engineering and Computer Science (Eurogen99) Jyväskylä, Finlandia,
- [6] Miller J. F., Thomson P., and Fogarty T. C., "Designing Electronic Circuits Using Evolutionary Algorithms. Arithmetic Circuits: A Case Study", in *Genetic Algorithms and Evolution Strategies in Engineering and Computer Science*: D. Quagliarella, J. Periaux, C. Poloni and G. Winter (eds), Wiley, 1997.
- [7] Heng Liu, Julian F. Miller, Andy M. Tyrrell. "Intrinsic Evolvable Hardware Implementation of a Robust Biological Development Model for Digital Systems," eh, pp. 87-92, 2005 NASA/DoD Conference on Evolvable Hardware (EH'05), 2005.
- [8] Synthesis of Digital Circuits with Genetic Algorithms: A Fractional-Order Approach Cecilia Reis, J. A. Tenreiro Machado, J. Boaventura Cunha ,Volume 1 Number 4 2004, *International Journal of Computational Intelligence* pp289-295
- [9] Evolutionary Design of Combinational Logic Circuits Cecilia Reis, J. A. Tenreiro Machado, and J. Boaventura Cunha, pp. 507-513 *Journal of Advanced Computational Intelligence and Intelligent Informatics*, Vol.8, No.5 pp. 507-513, 2004
- [10] B. Hounsell and T. Arslan, A Novel Evolvable Hardware Framework for the Evolution of High Performance Digital Circuits , *GECCO-2000*, 8-12 July 2000, pp. 525 - 532.
- [11] Phillip Moore, Ganesh K Venayagamoorthy. "Evolving Combinational Logic Circuits Using a Hybrid Quantum Evolution and Particle Swarm Inspired Algorithm," eh, pp. 97-102, 2005 NASA/DoD Conference on Evolvable Hardware (EH'05), 2005.
- [12] Coello Coello, Carlos A.; Christiansen, Alan D. and Hernández Aguirre, Arturo, Use of Evolutionary Techniques to Automate the Design of Combinational Circuits, *International Journal of Smart Engineering System Design*, Elsevier Science, Vol. 2, No. 4, pp. 299-314, June 2000